



Architettura degli elaboratori

Lezione 8 - teoria

Prof.ssa Valentina Ciriani

Università degli Studi di Milano

www.di.unimi.it/ciriani

1



Contenuto

1. **Assembler MIPS [PH 2.7, 2.5]**
 1. Le istruzioni di controllo [PH 2.7]
 2. Rappresentazione delle istruzioni nel calcolatore [PH 2.5]

-MKM = M. Morris Mano, C.R. Kime, T. Martin, Reti logiche, Pearson

-PH = D.A. Patterson, J.L. Hennessy, Struttura e Progetto dei Calcolatori, Zanichelli

2

2



Istruzioni di controllo

- istruzioni che alterano l'ordine di esecuzione:
 - la prossima istruzione da eseguire non è l'istruzione successiva all'istruzione corrente
- Permettono di eseguire cicli e condizioni
- In assembly MIPS le strutture di controllo sono molto semplici e primitive

3

3



Salto condizionato

- Istruzioni di salto condizionato (conditional branch):
 - il salto viene eseguito solo se una certa condizione risulta soddisfatta
- beq (branch on equal) e
- bne (branch on not equal)

```

beq $s1, $s2, L1 # vai a L1 se
                  # ($s1=$s2)
bne $s1, $s2, L1 # vai a L1 se
                  # ($s1≠$s2)

```

4

4



Salto incondizionato

- salto incondizionato (unconditional branch):
 - il salto viene eseguito sempre

```
j L1 # vai a L1
```

5

5



Esempio if ... then

```
f in $s0
g in $s1
```

```
if (i==j) {f=g+h}
```

codice modificato:

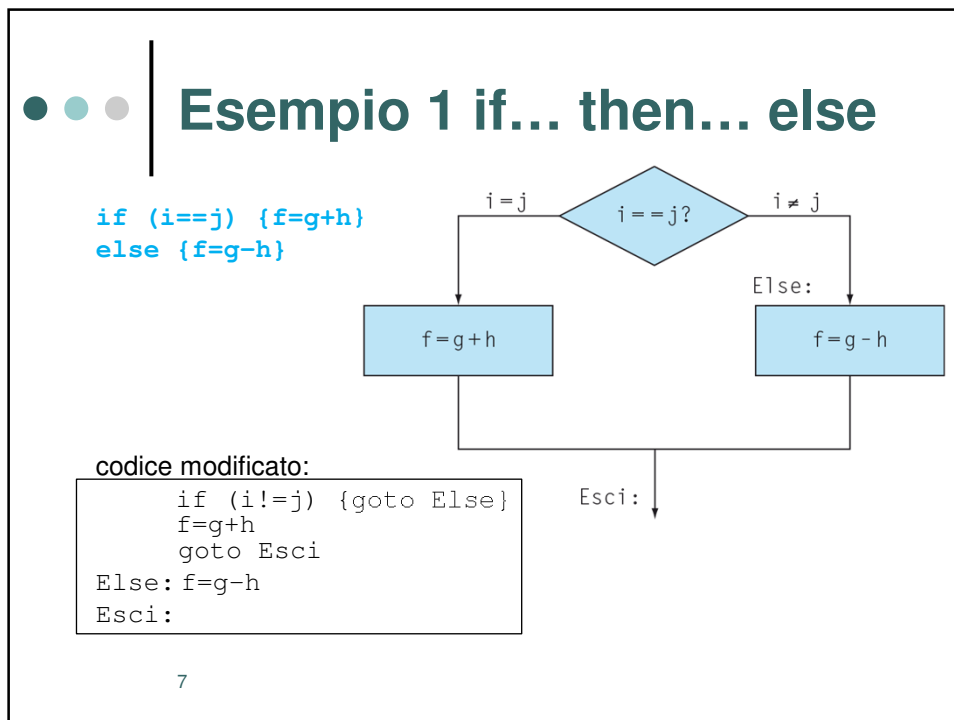
```
h in $s2
i in $s3
j in $s4
```

```
if i!=j {goto Esci}
f=g+h
Esci:
```

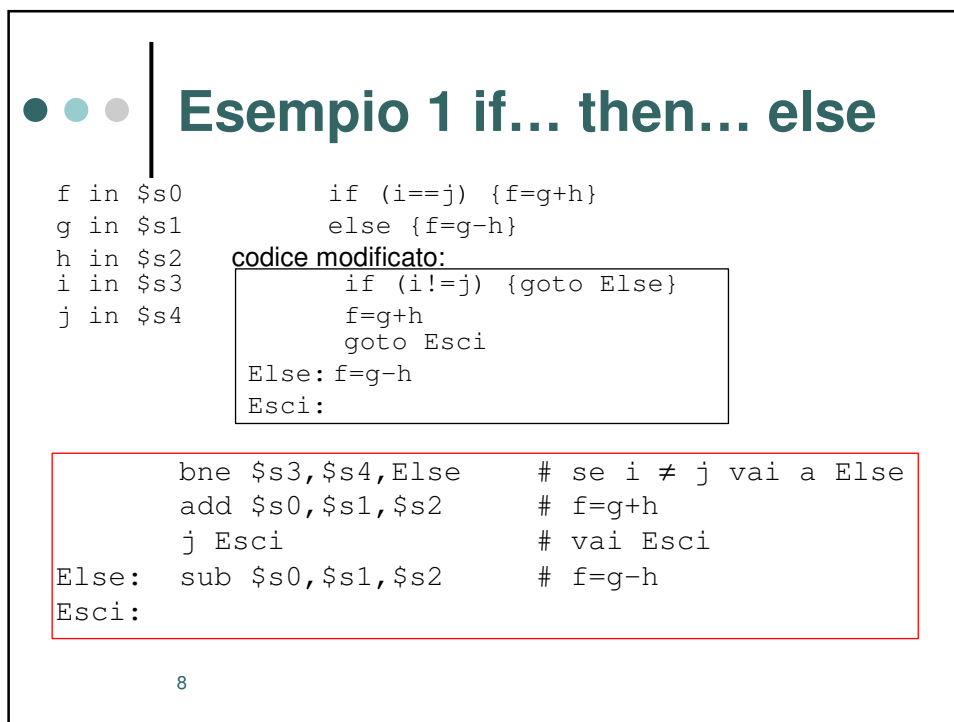
```
bne $s3, $s4, Esci # se i!=j vai a Esci
add $s0, $s1, $s2 # se i=j f=g+h
Esci:
```

6

6



7



8



Esempio 2 if... then... else

```
f in $s0      if (i==j) {f=g+h}
g in $s1      else {f=g-h}
h in $s2
i in $s3
j in $s4
```

codice modificato:

```
if (i==j) {goto Then}
f=g-h
goto Esci
Then: f=g+h
Esci:
```

```
beq $s3,$s4,Then    # se i=j vai a Then
sub $s0,$s1,$s2     # f=g-h
j Esci              # vai a Esci
Then: add $s0,$s1,$s2 # f=g+h
Esci:
```

9

9



Esempio 1 for

```
i in $s3      for (i!=j) {i=i+1}
j in $s4
```

codice modificato:

```
Loop: if (i==j) {goto Esci}
      i=i+1
      goto Loop
Esci:
```

```
Loop: beq $s3, $s4, Esci # se i=j vai a Esci
      addi $s3, $s3, 1   # i=i+1
      j Loop            # vai a Loop
Esci:
```

10

10



Esempio 2 for

```
h in $s2
i in $s3
j in $s4
```

```
for (i:=0; i!=j; i++) {h=h-1}
```

codice modificato:

```
i=0
Loop: if (i==j) {goto Esci}
      h=h-1
      i=i+1
      goto Loop
Esci:
```

```
add $s3, $zero, $zero # i=0
Loop: beq $s3, $s4, Esci # se i=j vai a Esci
      addi $s2, $s2, -1 # h=h-1
      addi $s3, $s3, 1 # i=i+1
      j Loop # vai a Loop
Esci:
```

11

11



minore e maggiore

```
slt $t0, $s0, $s1 # set on less than
```

• se \$s0 < \$s1 allora \$t0=1

• se \$s0 >= \$s1 allora \$t0=0

Con `slt`, `beq` e `bne` si possono implementare tutti i test sui valori di due variabili

(==, !=, <, <=, >, >=)

12

12



minore e maggiore immediato

```
slti $t0, $s0, 100 # set on less than
```

○ se $s0 < 100$ allora $t0=1$

○ se $s0 \geq 100$ allora $t0=0$

13

13



Esempio if... then... else

```
f in $s0          if (i<j){f=g+h}
```

```
g in $s1          else {f=g-h}
```

```
h in $s2          codice modificato:
```

```
i in $s3          if (i>=j){goto Else}
```

```
j in $s4          f=g+h
```

```
                  goto Esci
```

```
Else: f=g-h
```

```
Esci:
```

```
slt $t0, $s3, $s4      # se i<j $t0=1
```

```
beq $t0, $zero, Else  # se i>=j vai a Else
```

```
add $s0, $s1, $s2      # f=g+h
```

```
j Esci                 # vai a Esci
```

```
Else: sub $s0, $s1, $s2 # f=g-h
```

```
Esci:
```

14

14



Esempio for

```
h in $s2
i in $s3
j in $s4
```

for (i:=0; i<j; i++) {h=h-1}
codice modificato:

```
i=0
Loop: if (i>=j){goto Esci}
      h=h-1
      i=i+1
      goto Loop
Esci:
```

```
add $s3, $zero, $zero # i=0
Loop: slt $t0, $s3, $s4 # se i>=j $t0=0
      beq $t0, $zero, Esci # se i>=j vai a Esci
      addi $s2, $s2, -1 # h=h-1
      addi $s3, $s3, 1 # i=i+1
      j Loop # vai a Loop
Esci:
```

15

15



Linguaggio macchina

- Le istruzioni assembly devono essere tradotte in linguaggio macchina (sequenze di 0 e 1) per poter essere eseguite
- Le istruzioni in linguaggio macchina sono **lunghe 32 bit** (come i registri e le parole di memoria)
- Nelle istruzioni macchina i **registri** sono indirizzati con numeri progressivi da **0 a 31**

16

16



Registri

Nome **Numero** **Binario** (5 bit: 00000-11111)

\$zero **0** **00000**

\$at 1 (registro riservato assemblatore)

\$v0, \$v1 2 - 3

\$a0-\$a3 4 - 7

\$t0-\$t7 **8 - 15** **01000 - 01111**

\$s0-\$s7 **16 - 23** **10000 - 10111**

\$t8, \$t9 **24, 25** **11000 - 11001**

\$k0, \$k1 26 - 27 (registri riservati sistema oper.)

\$gp, \$sp 28 - 29

\$fp, \$ra 30 - 31

17

17



Formato istruzioni

- Le istruzioni macchina hanno una rappresentazione di 32 bit
- Una istruzione macchina (32 bit) è suddivisa in gruppi di bit detti **campi**
- Il numero e la lunghezza dei campi di una istruzione determinano il **formato** dell'istruzione
- Ci sono 3 tipi di formato: R, I e J

	6 bit	5 bit	5 bit	5 bit	5 bit	6 bit
R	op	rs	rt	rd	shamt	funct
I	op	rs	rt	indirizzo/costante		
J	op	indirizzo/costante				

31

0

31

0

18

18



Formato istruzioni

- Formato **R** (registro)
 - Istruzioni aritmetiche logiche e scorrimento
- Formato **I** (immediato)
 - Istruzioni aritmetiche logiche con operandi immediati
 - Istruzioni di trasferimento dati
 - Istruzioni di salto condizionato
- Formato **J** (jump)
 - Istruzioni di salto incondizionato

19

19



Formato R (registro)

op	rs	rt	rd	shamt	funct
6 bit	5 bit	5 bit	5 bit	5 bit	6 bit

Campi:

- **op**: (opcode) identifica la classe di istruzione, nel formato R op = 000000
- **rs**: registro contenente il primo operando sorgente
- **rt**: registro contenente il secondo operando sorgente
- **rd**: registro destinazione contenente il risultato
- **shamt**: shift amount (scorrimento)
- **funct**: indica la variante specifica dell'operazione

20

20



Istruzioni di formato R

- Istruzioni con formato di tipo R:
 - add, sub, mult, div (aritmetiche)
 - sll, srl (shift, scorrimento)
 - and, or, nor, slt (logiche)
- Le diverse istruzioni si distinguono tra loro in base al campo funct
- op è sempre 000000

21

21



Esempi

	6 bit	5 bit	5 bit	5 bit	5 bit	6 bit
	op	rs	rt	rd	shamt	func
add \$s1, \$s2, \$s3	000000	10010	10011	10001	00000	100000
sub \$t0, \$t2, \$t3	000000	01010	01011	01000	00000	100010
sll \$t9, \$s3, 4	000000	10011	00000	11001	00100	000000
srl \$t0, \$s4, 31	000000	10100	00000	01000	11111	000010
and \$s0, \$t1, \$t7	000000	01001	01111	10000	00000	100100
or \$s5, \$t2, \$t6	000000	01010	01110	10101	00000	100101
nor \$t0, \$t4, \$t8	000000	01100	11000	01000	00000	100111
mult \$t5, \$s6	000000	01101	10110	00000	00000	011000
div \$t2, \$s7	000000	01010	10111	00000	00000	011010
slt \$t0, \$t4, \$t8	000000	01100	11000	01000	00000	101010

22

22



Esempio in esadecimale

	6 bit	5 bit	5 bit	5 bit	5 bit	6 bit
	op	rs	rt	rd	shamt	func
add \$s1, \$s2, \$s3	000000	10010	10011	10001	00000	100000

add \$s1, \$s2, \$s3
 0000 0010 0101 0011 1000 1000 0010 0000
 0 2 5 3 8 8 2 0

In esadecimale: 0x02538820

23

23



Formato I (immediato)

op	rs	rt	indirizzo/costante
6 bit	5 bit	5 bit	16 bit

Campi:

- o **op**: (opcode) identifica il tipo di istruzione
- o **rs**: registro base
- o **rt**: registro destinazione
- o **indirizzo**: $[-2^{15}, 2^{15} - 1]$, può essere:
 - lo spiazzamento (offset) rispetto all'indirizzo base
 - una costante
 - l'etichetta di un salto

24

24



Istruzioni di formato I

- Istruzioni formato di tipo I:
 - `addi` (aritmetiche immediate)
 - `andi`, `ori`, `slti` (logiche immediate)
 - `beq`, `bne` (salti condizionati)
 - `lw`, `sw` (accessi in memoria)
- Le diverse istruzioni si distinguono tra loro in base al campo `op`

25

25



Esempi

	6 bit	5 bit	5 bit	16 bit
	op	rs	rt	Indirizzo/costante
<code>addi \$s2, \$s3, -29</code>	001000	10011	10010	1111111111100011
<code>andi \$s2, \$s3, 37</code>	001100	10011	10010	000000000100101
<code>ori \$s2, \$s3, -1</code>	001101	10011	10010	1111111111111111
<code>beq \$s2, \$s3, L1</code>	000100	10010	10011	0000000000000010
<code>bne \$s2, \$s3, L2</code>	000101	10010	10011	1111111111111101
<code>lw \$s2, 8(\$s3)</code>	100011	10011	10010	000000000001000
<code>sw \$s2, 8(\$s3)</code>	101011	10011	10010	000000000001000
<code>slti \$s2, \$s3, 37</code>	001010	10011	10010	000000000100101

26

26



Istruzioni di salto

salta 2 in basso



<code>beq \$s2, \$s3, L1</code>	000100	10010	10011	0000000000000010
---------------------------------	--------	-------	-------	------------------

- Il valore dell'etichetta L1 (indirizzo di destinazione del salto) è **relativo** al **Program Counter** (PC)
- il PC contiene l'indirizzo della prossima istruzione da eseguire
- L'indirizzamento relativo al PC permette di fare dei salti condizionati a locazioni di memoria il cui indirizzo assoluto non è esprimibile con soli 16-bit
- L'assemblatore sostituisce l'etichetta L1 con **l'indirizzo relativo** a PC: quante istruzioni saltare
- L1 può essere negativo per i salti all'indietro

27

27



Esempio

<code>beq \$s2, \$s3, L1</code>	000100	10010	10011	0000000000000010
---------------------------------	--------	-------	-------	------------------

$PC+4 = 40016$

perché, quando si esegue un'istruzione, il valore successivo del PC (ovvero PC+4) è già stato calcolato

<code>beq \$s2, \$s3, L1</code>	40012: 00010010010100110000000000000010
<code>add \$t0, \$t3, \$t4</code>	40016: ...
<code>sub \$t3, \$t3, \$s4</code>	40020:
<code>L1: ...</code>	40024:

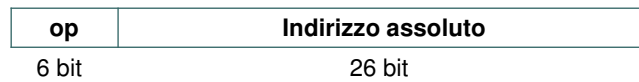
- $(IA - (PC+4))/4 = (40024 - 40016)/4 = 2$
- Indirizzo assoluto del salto:
 $IA = (PC+4) + L1 \times 4 = 40016 + 2 \times 4 = 40024$

28

28



Formato J (jump)



- Istruzione formato di tipo J:
 - j L1 (salto incondizionato)

Campi:

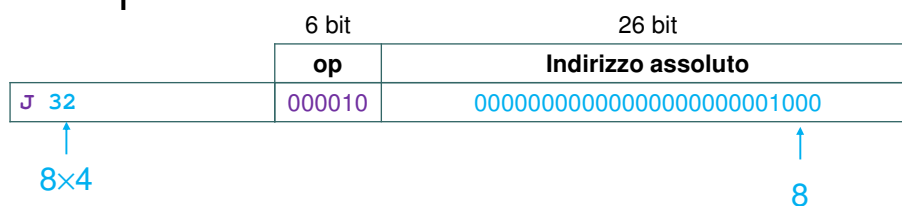
- op**: (opcode) identifica il tipo di istruzione
- indirizzo**: contiene l'indirizzo assoluto per la destinazione del salto come indirizzo di parola

29

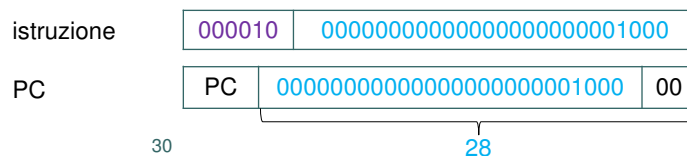
29



Esempio



- I 26-bit di indirizzo nelle jump sono un indirizzo di word ovvero un indirizzo di byte composto da 28-bit (si divide per 4)



30



Salti incondizionati lunghi

Per saltare ad indirizzi superiori a 2^{28} Byte si usa l'istruzione:

`jr rs` (jump register con formato R)

Salta all'indirizzo di memoria assoluto contenuto nel registro rs (spazio di 2^{32} byte cioè 2^{30} word: l'intero spazio di memoria)

31

31



Salti condizionati lunghi

Se voglio fare un salto condizionato ad un indirizzo relativo che non è nell'intervallo: $[-2^{15}, 2^{15} - 1]$:

`beq $s0, $s1, L1`

viene trasformata in:

`bne $s0, $s1, L2`

`j L1`

(indirizzo assoluto in linguaggio macchina)

`L2:`

32

32



Esercizi da fare a casa 1

1. Si supponga che `$t0` contenga il valore `0x00101000`. Determinare il contenuto di `$t2` dopo l'esecuzione delle seguenti istruzioni:

```

    slt $t2, $zero, $t0
    bne $t2, $zero, ELSE
    j FINE
ELSE: addi $t2, $t2, 2
FINE:

```

33

33



Esercizi da fare a casa 2

2. Si supponga che il prossimo valore del program counter (`PC+4`) sia `0x20000000`
 - a) Qual è l'insieme degli indirizzi che il PC può assumere dopo che è stata eseguita l'istruzione di jump incondizionata (`j`)?
 - b) Quale intervallo di indirizzi si può raggiungere utilizzando l'istruzione MIPS `beq`?

34

34



Esercizi da fare a casa 3

- Tradurre le seguenti istruzioni in linguaggio macchina

```
addi $t0, $s6, 4
add  $t1, $s6, $s0
sw   $t1, 0($t0)
lw   $t0, 0($t0)
add  $s0, $t1, $t0
```

35

35



Esercizi da fare a casa 4

- Tradurre in notazione esadecimale la seguente istruzione assembler:
`sw $t1, 32($t2)`
 e determinare il suo formato
- Scrivere l'istruzione assembler associata ai seguenti campi del MIPS: $op = 0$, $rs = 8$, $rt = 9$, $rd = 8$, $shamt = 0$, $funz = 34$

36

36



Esercizi da fare a casa 5

6. Scrivere l'istruzione assembler associata ai seguenti campi del MIPS: $op = 0x23$, $rs = 9$, $rt = 12$, $costante = 0x4$ individuare il formato e scrivere la rappresentazione binaria

37

37



Esercizi da fare a casa 6

7. Si consideri una nuova istruzione chiamata `rpt`. Questa istruzione combina in un'unica istruzione la verifica di una condizione di fine ciclo con il decremento dell'indice del ciclo. Per esempio,
`rpt $s0, ciclo`
 corrisponde al seguente frammento di codice (con x in $\$s0$):

```
if x > 0 {
    x = x - 1
    goto ciclo}
```

 - a) Quale sarebbe il formato di istruzione più appropriato per implementare l'istruzione (R, I o J)?
 - b) Determinare una sequenza di istruzioni MIPS che consenta di implementare l'istruzione riportata sopra

38

38



Esercizi da fare a casa 7

8. Si consideri il seguente ciclo in assembler MIPS:

```
CICLO: slt  $t2, $zero, $t1
       beq  $t2, $zero, FINE
       addi $t1, $t1, -1
       addi $s2, $s2, 2
       j    CICLO

FINE:
```

- Supponendo che `$t1` sia inizializzato a 10 e `$s2` a 0. Quale è il valore finale di `$s2`?
- Scrivere il codice in un linguaggio ad alto livello equivalente al ciclo assembler riportato sopra. Si supponga che i registri `$s1, $s2, $t1` e `$t2` contengano rispettivamente le variabili intere `A, B, i` e `temp`.

39

39



Esercizi da fare a casa 8

9. Tradurre il seguente frammento di codice in codice assembler MIPS. Si supponga che le variabili `a`, `b`, `i` e `j` siano contenute rispettivamente nei registri `$s0`, `$s1`, `$t0` e `$t1`. Si supponga che il registro `$s2` contenga l'indirizzo base del vettore `D`

```
for i=0; i<a; i++
{for j=0; j<b; j++
 {D[4*j] = i+j}}
```

40

40



Esercizi da fare a casa 9

10. Tradurre il seguente ciclo un linguaggio ad alto livello. Si supponga che il registro `$t1` contenga la variabile intera `i`, `$s2` contenga la variabile intera `Ris` e `$s0` l'indirizzo base del vettore `MemVett`.

```

        addi $t1, $zero, 0
CICLO: lw  $s1, 0($s0)
        add  $s2, $s2, $s1
        addi $s0, $s0, 4
        addi $t1, $t1, 1
        slti $t2, $t1, 100
        bne $t2, $zero, CICLO

```

41

41



Architettura degli elaboratori

Lezione 8 - esercizi

Prof.ssa Valentina Ciriani

Università degli Studi di Milano

www.di.unimi.it/ciriani

42



Esercizio 1

Qual è il codice assembler MIPS corrispondente alla seguente istruzione? Si supponga che le variabili f , g e h siano date e che possano essere dichiarate come interi a 32 bit all'interno del programma: $f = g + (h - 5)$

```
addi f, h, -5
add  f, f, g
```

Se f , g , h sono in $\$s0$, $\$s1$ e $\$s2$ allora:

```
addi $t0, $s2, -5
add  $s0, $t0, $s1
```

43

43



Esercizio 2

Scrivere una sola istruzione in un linguaggio ad alto livello, che corrisponda alle seguenti istruzioni MIPS.

```
add f, g, h
add f, i, f
```

$f = g + h + i$

44

44



Esercizio 3.a

Scrivere il codice assembler MIPS che corrisponde alla seguente istruzione. Si supponga che le variabili i e j , siano assegnate rispettivamente ai registri $\$s3$ e $\$s4$. Si assuma che l'indirizzo base dei vettori A e B sia contenuto rispettivamente nei registri $\$s6$ e $\$s7$:

$B[8] = A[i-j]$

```
sub $t0, $s3, $s4      # i-j
sll $t0, $t0, 2        # (i-j)*4
add $t0, $s6, $t0      # indirizzo di A[i-j]
lw  $t1, 0($t0)        # $t1 contiene A[i-j]
sw  $t1, 32($s7)       # B[8] = A[i-j]
```

45

45



Esercizio 3.b

Scrivere il codice assembler MIPS che corrisponde alla seguente istruzione. Si supponga che le variabili i e j , siano assegnate rispettivamente ai registri $\$s3$ e $\$s4$. Si assuma che l'indirizzo base dei vettori A e B sia contenuto rispettivamente nei registri $\$s6$ e $\$s7$ $B[8] = A[i] + A[j]$

```
sll $t0, $s3, 2        # i*4
add $t0, $s6, $t0      # indirizzo di A[i]
lw  $t1, 0($t0)        # $t1 contiene A[i]
sll $t0, $s4, 2        # j*4
add $t0, $s6, $t0      # indirizzo di A[j]
lw  $t2, 0($t0)        # $t2 contiene A[j]
add $t0, $t1, $t2      # A[i] + A[j]
sw  $t0, 32($s7)       # B[8] = A[i] + A[j]
```

46

46



Esercizio 4

4. Tradurre il seguente spezzone di codice assembler MIPS in un linguaggio ad alto livello. Si supponga che le variabili *f*, *g*, *h*, *i* e *j* siano assegnate rispettivamente ai registri *\$s0*, *\$s1*, *\$s2*, *\$s3* e *\$s4*. Si assuma che l'indirizzo base (&) dei vettori *A* e *B* sia contenuto nei registri *\$s6* e *\$s7* rispettivamente

```
addi $t0, $s6, 4      # $t0 cont. &A[1]
add  $t1, $s6, $zero # $t1 contiene &A[0]
sw   $t1, 0($t0)      # &A[0] è in A[1]
lw   $t0, 0($t0)      # $t0 contiene &A[0]
add  $s0, $t1, $t0     # f = &A[0] + &A[0]
```

codice: *A[1]=&A[0]; f = 2*(A[1])*

47

47



Esercizio 5

Supponendo che *\$t0* = 0xAAAAAAAA, *\$t1* = 0x12345678 determinare il contenuto di *\$t2* dopo l'esecuzione delle seguenti istruzioni:

```
sll $t2, $t0, 4
or  $t2, $t2, $t1
```

```
$t0: 1010 1010 1010 1010 1010 1010 1010 1010
$t1: 0001 0010 0011 0100 0101 0110 0111 1000
sll $t2, $t0, 4
$t2: 1010 1010 1010 1010 1010 1010 1010 0000
or  $t2, $t2, $t1
$t2: 1011 1010 1011 1110 1111 1110 1111 1000
$t2 = 0xBABEFEF8
```

48

48



Esercizio 6

Supponendo che $\$t0 = 0xAAAAAAAA$,
 $\$t1 = 0x12345678$ determinare il contenuto di $\$t2$ dopo
 l'esecuzione delle seguenti istruzioni:

```
srl $t2, $t0, 3
andi $t2, $t2, 0xFFEF
```

```
$t0:      1010 1010 1010 1010 1010 1010 1010 1010
0xFFEF16: 0000 0000 0000 0000 1111 1111 1110 1111
srl $t2, $t0, 3
$t2:      0001 0101 0101 0101 0101 0101 0101 0101
andi $t2, $t2, 0xFFEF
$t2:      0000 0000 0000 0000 0101 0101 0100 0101
$t2 = 0x00005545
```

49

49



Esercizio 7

Determinare la sequenza più corta di istruzioni MIPS
 che consente di estrarre i bit da 11 a 16 dal registro
 $\$t0$ e li sostituisce ai bit da 26 a 31 del registro $\$t1$
 senza modificare gli altri bit dei registri $\$t0$ e $\$t1$
 (provare il vostro codice con $\$t0 = 0$ e $\$t1 =$
 $0xFFFFFFFF$: l'esecuzione potrebbe rivelare una svista
 comune)

```
$t0 = t31 t30 t29 t28 t27 t26 t25 t24 t23 t22 t21 t20 t19 t18 t17 t16 t15 t14 t13 t12 t11 t10 t9 t8 t7 t6 t5 t4 t3 t2 t1 t0
$t1 = i31 i30 i29 i28 i27 i26 i25 i24 i23 i22 i21 i20 i19 i18 i17 i16 i15 i14 i13 i12 i11 i10 i9 i8 i7 i6 i5 i4 i3 i2 i1 i0

$t1 = t16 t15 t14 t13 t12 t11 i25 i24 i23 i22 i21 i20 i19 i18 i17 i16 i15 i14 i13 i12 i11 i10 i9 i8 i7 i6 i5 i4 i3 i2 i1 i0
```

50

50



Esercizio 7

```

$t0 = t31 t30 t29 t28 t27 t26 t25 t24 t23 t22 t21 t20 t19 t18 t17 t16 t15 t14 t13 t12 t11 t10 t9 t8 t7 t6 t5 t4 t3 t2 t1 t0
$t1 = i31 i30 i29 i28 i27 i26 i25 i24 i23 i22 i21 i20 i19 i18 i17 i16 i15 i14 i13 i12 i11 i10 i9 i8 i7 i6 i5 i4 i3 i2 i1 i0

$t1 = t16 t15 t14 t13 t12 t11 i25 i24 i23 i22 i21 i20 i19 i18 i17 i16 i15 i14 i13 i12 i11 i10 i9 i8 i7 i6 i5 i4 i3 i2 i1 i0

```

Idea:

- 1) sposto i bit da 11 a 16 di \$t0 in posizione da 26 a 31 e metto tutti gli altri a 0

```
$t0 = t16 t15 t14 t13 t12 t11 00 0000 0000 0000 0000 0000 0000
```

- 2) preparo una maschera per \$t1:

```
$t2 = 0000 0011 1111 1111 1111 1111 1111 1111
```

- 3) faccio l'AND tra la maschera e \$t1:

```
$t1 = 0000 00 i25 i24 i23 i22 i21 i20 i19 i18 i17 i16 i15 i14 i13 i12 i11 i10 i9 i8 i7 i6 i5 i4 i3 i2 i1 i0
```

- 4) faccio l'OR tra \$t1 e \$t0:

```
$t1 = t16 t15 t14 t13 t12 t11 i25 i24 i23 i22 i21 i20 i19 i18 i17 i16 i15 i14 i13 i12 i11 i10 i9 i8 i7 i6 i5 i4 i3 i2 i1 i0
```

51

51



Esercizio 7

Determinare una di istruzioni MIPS che consente di estrarre i bit da 11 a 16 dal registro \$t0 e li sostituisce ai bit da 26 a 31 del registro \$t1 senza modificare gli altri bit dei registri \$t0 e \$t1 (provare il vostro codice con \$t0 = 0 e \$t1 = 0xFFFFFFFF: l'esecuzione potrebbe rivelare una svista comune)

- 1) {


```

srl $t3, $t0, 11      # tolgo i bit 0-10 di $t0
sll $t3, $t3, 26      # sposto i bit tra 11 a 16
                      # in posizione 26 31

```
- 2) {


```

ori $t2, $zero, 0x03FF # inserisco i primi 10 uni
sll $t2, $t2, 16       # li sposto a sinistra di 16 bit
ori $t2, $t2, 0xFFFF  # inserisco gli ultimi 16 uni

```
- 3) and \$t1, \$t1, \$t2 # metto a zero i bit tra 26 a 31
- 4) or \$t1, \$t1, \$t3 # metto i bit calcolati in posizione tra 26 a 31

52

52

Esercizio 7

```
$t0= 1111 1111 1111 1110 0010 0111 1111 1111
$t1= 1111 1111 1111 1111 1111 1111 1111 1111
$t1= 0001 0011 1111 1111 1111 1111 1111 1111
```

Eseguo il codice :

1) sposto i bit da 11 a 16 di \$t0 in posizione da 26 a 31 e metto tutti gli altri a 0

```
srl $t3, $t0, 11 # tolgo i bit inferiori a 11
$t3 = 0000 0000 0001 1111 1111 1111 1100 0100
sll $t3, $t3, 26 # sposto i bit tra 11 a 16 in
posizione 26 31
$t3 = 0001 0000 0000 0000 0000 0000 0000 0000
```

2) preparo una maschera per \$t1:

```
ori $t2, $zero, 0x03FF #inserisco i primi 10 uni
$t2 = 0000 0000 0000 0000 0000 0011 1111 1111
sll $t2, $t2, 16 # li sposto a sinistra di 16 bit
$t2 = 0000 0011 1111 1111 0000 0000 0000 0000
ori $t2, $t2, 0xFFFF # inserisco gli ultimi 16 uni
$t2 = 0000 0011 1111 1111 1111 1111 1111 1111
```

3) faccio l'AND tra la maschera e \$t1:

```
$t1= 0000 0011 1111 1111 1111 1111 1111 1111
```

4) faccio l'OR tra \$t1 e \$t3:

```
53 $t1= 0001 0011 1111 1111 1111 1111 1111 1111
```

53

Esercizio 7 altra soluzione

Determinare una di istruzioni MIPS che consente di estrarre i bit da 11 a 16 dal registro \$t0 e li sostituisce ai bit da 26 a 31 del registro \$t1 senza modificare gli altri bit dei registri \$t0 e \$t1 (provare il vostro codice con \$t0 = 0 e \$t1 = 0xFFFFFFFF: l'esecuzione potrebbe rivelare una svista comune)

- 1) { srl \$t3, \$t0, 11 # tolgo i bit 0-10 di \$t0
sll \$t3, \$t3, 26 # sposto i bit tra 11 a 16
in posizione 26 31
- 2) { sll \$t1, \$t1, 6 # sposto a sinistra di 6 bit
srl \$t1, \$t1, 6 # sposto a destra di 6 bit
- 3) or \$t1, \$t1, \$t3 # metto i bit calcolati in
posizione tra 26 a 31

54

54

Esercizio 7 altra soluzione

```
$t0= 1111 1111 1111 1110 0010 0111 1111 1111
$t1= 1111 1111 1111 1111 1111 1111 1111 1111
$t1= 0001 0011 1111 1111 1111 1111 1111 1111
```

Eseguo il codice :

1) sposto i bit da 11 a 16 di \$t0 in posizione da 26 a 31 e metto tutti gli altri a 0

```
srl $t3, $t0, 11 # tolgo i bit inferiori a 11
$t3 = 0000 0000 0001 1111 1111 1111 1100 0100
sll $t3, $t3, 26 # sposto i bit tra 11 a 16 in
posizione 26 31
$t3 = 0001 0000 0000 0000 0000 0000 0000 0000
```

2) metto a zero i 6 bit più significativi di \$t1:

```
sll $t1, $t1, 6 # sposto a sinistra di 6 bit
$t1 = 1111 1111 1111 1111 1111 1111 0000 0000
srl $t1, $t1, 6 # sposto a destra di 6 bit
$t1 = 0000 0011 1111 1111 1111 1111 1111 1111
```

3) faccio l'OR tra \$t1 e \$t3:

```
or $t1, $t1, $t3
$t1= 0001 0011 1111 1111 1111 1111 1111 1111
```

55

55

Esercizio 8

Determinare la sequenza più corta di istruzioni MIPS che si possono utilizzare per mettere in \$t2 il NOT del contenuto di \$t1

```
nor $t2, $t1, $t1
```

56

56



Esercizio 9

Determinare una sequenza di istruzioni MIPS che si possono utilizzare per tradurre la seguente istruzione:

$A = C[0] \ll 4$

Si supponga che $\$t1 = A$, $\$t2 = B$ e $\$s1$ contenga l'indirizzo base di C

```
lw $t0, 0($s1)
```

```
sll $t1, $t0, 4
```

57

57